

Praktikumsbericht

<Tejani Awudu Mohammed>

1. Juni 2026

Inhaltsverzeichnis

1	Praktikumsbericht	2
1.1	19.05.26 – Dienstag	2
1.2	20.05.26 – Mittwoch	2
1.3	21.05.26 – Donnerstag	2
1.4	22.05.26 – Freitag	2
1.5	26.06.26 – Dienstag	3
1.6	27.06.26 – Mittwoch	3
1.7	28.06.26 – Donnerstag	3
1.8	29.05.26 – Freitag	4
2	LinkedList	4
2.1	Singly LinkedList	4
2.2	Doubly LinkedList	4
2.2.1	Doubly LinkedList	4
3	Neo4j	5
4	LaTeX	5

1 Praktikumsbericht

1.1 19.05.26 – Dienstag

Nach dem Gespräch habe ich mir bereits Gedanken darüber gemacht, wie das Frontend gestaltet werden soll.

1.2 20.05.26 – Mittwoch

Ich habe zunächst alle vertraglichen Angelegenheiten bis 13 Uhr geregelt. Danach bekam ich die Anweisung, mir die Repositories im Raw-Format anzusehen, um die Header zu verstehen und ein besseres Gefühl dafür zu bekommen, wie alles funktioniert und zusammenhängt. Dafür habe ich mir die wichtigsten Inhalte mit Unterstützung von KI erklären lassen, um schnell erste Zusammenhänge zu verstehen und offene Fragen zu klären.

- STEMgraph abfragen

1.3 21.05.26 – Donnerstag

Mein Auftrag war es, mich weiter zu vertiefen. Ich sollte mir bei `LinkedList` die Pointer `next_ptr` und `prev_ptr` in C ansehen und ein Verständnis dafür aufbauen. Dabei habe ich mit KI einige Punkte erarbeitet:

- Was ich unter dem STEMgraph-Konzept verstehe.
- Was ich beim Lesen der Raws beachten sollte.
- Welchen Zusammenhang die Metadaten haben.
- Welche relevanten Punkte es bei einer `LinkedList` gibt.
- Welche Arten einer `LinkedList` es gibt.
- Wie eine `Singly LinkedList` und eine `Doubly LinkedList` aufgebaut sind.
- Welche Unterschiede sie haben.
- Welchen Vorteil sie gegenüber einer `ArrayList` haben.

Außerdem sollte ich mir ein Tutorial zu Neo4j ansehen und mir die Frage stellen, warum `LinkedList` und Neo4j sich ähneln. Ich habe mir zwei bis drei Videos angeschaut und direkt festgestellt, dass der Name die Datei ist und der Array die Beziehung darstellt – grob gesagt. Was Neo4j genau ist und wie es funktioniert, schaue ich mir am nächsten Tag weiter an.

1.4 22.05.26 – Freitag

Ich sollte mit dem Bericht starten und beschreiben, womit ich mich die Woche beschäftigt hatte, welche relevanten Punkte mir aufgefallen sind und welche Fragen ich bisher noch nicht klären konnte. Dazu sollte ich ein Repository erstellen und eine Datei im LaTeX-Format hineinschreiben, aber noch nichts ausführen, sondern mir zunächst nur die LaTeX-Syntax ansehen. Beim weiteren Erfragen von Neo4j und LaTeX sind mir folgende Fragen aufgekommen:

- Wie bekomme ich die Dateien in den Wert der Eigenschaften?
- Inwiefern ist es hilfreich, die Challenges in LaTeX zu schreiben, um sie in Neo4j aufzurufen?

1.5 26.06.26 – Dienstag

Ich sollte meinen bisherigen Bericht in LaTeX mit Makefile schreiben. Wenn das erledigt ist, soll ich das Repository committen und einen Link schicken. Zusätzlich sollte ich, wenn ich fertig bin, ein Bash-Script schreiben, das ich einem Challenge-Repository gebe und das dann rekursiv alle weiteren Repositories herunterlädt und darauf aufbaut, was ich übergeben habe.

- Um den Bericht in LaTeX zu bekommen, musste ich zunächst die LaTeX-Distribution MacTeX und die VS-Code-Extension LaTeX Workshop installieren und einrichten.
- Als das erledigt war, musste ich Homebrew, den Paketmanager für Linux und macOS, installieren.
- Um dann Makefile mit GNU Make zu bekommen, musste das Terminal berechtigt werden, damit ich `make` über Homebrew im Terminal installieren kann. Zusätzlich brauchte ich noch die VS-Code-Extension Makefile Tools.
- Teilweise hatte ich dann schon mit dem Umbau in LaTeX begonnen.

1.6 27.06.26 – Mittwoch

Ich hatte damit begonnen, den gesamten Bericht in LaTeX zu formatieren und die Makefile zu befüllen.

- Um nicht alle Befehle selbst eingeben zu müssen.
- Ich habe mich zusätzlich damit auseinandergesetzt, um mehr Verständnis dafür zu bekommen und es etwas schöner aussehen zu lassen.

1.7 28.06.26 – Donnerstag

Habe ich weiter gemacht mit dem Bericht. Hatte mir Gedanke dazu gemacht wie ich am besten LaTeX in das Projekt einbinden sollte.

- Ich dachte mir vielleicht sollte ich eine LaTeX mit einbinden in das Projekt um vielleicht direkt denn Inhalt für die Node daraus zulesen und nicht die md Variante nutzen.
- Da ist mir dann aufgefallen das LaTeX ja nur sinn machen würde um es dann auf der Website download bar zu machen.

Dann sollte ich schon mal das Bash-Script aufsetzten um alle Repo's nach dem ersten rekursiv zu downloaden.

- hatte mir dazu angeguckt für was man die Bash-Scripte nutzt bzw. was sie sind und welche Funktionen sie haben bzw. haben könnten

1.8 29.05.26 – Freitag

Sollte ich dann weiter machen mit dem Bash-Script hatte mir dann etwas Verständnis für das Konzept aufgebaut und die Logik der Abhängigkeiten etwas geklärt.

- gucke mir dann am Montag an wie ich das Script und die Pipeline umsetzte

Parallel dazu hatte ich weiter an der LaTeX- und Berichtstruktur gearbeitet.

Dann musste ich zur Firma mir einen Laptop abholen und in mit Linux aufsetzen und mir alles nötige beschaffen was ich brauch um vernünftig zuarbeiten

2 LinkedList

Eine LinkedList ist eine Datenstruktur, die aus Knoten besteht, die miteinander verbunden sind. Jeder Knoten enthält Daten und einen Verweis auf den nächsten Knoten in der Liste. Es gibt verschiedene Arten von LinkedLists, darunter Singly LinkedLists und Doubly LinkedLists.

2.1 Singly LinkedList

Die Singly LinkedList ist die einfachste Variante, bei der jedes Element auf das nächste Element zeigt

```
// Singly LinkedList
struct node{
int data;
struct node* link;
};
```

2.2 Doubly LinkedList

Die Doubly LinkedList ist die Variante, bei der das erste Element mit Null startet, dann die Data kommt und zusätzlich Next und Prev vorhanden sind. Dadurch werden einige Dinge ermöglicht.

```
// Doubly LinkedList
struct node{
struct node* prev;
int data;
struct node* next;
};
```

2.2.1 Doubly LinkedList

Die Doubly Linked List ist deutlich effizienter als eine Array List und braucht weniger Rechenleistung. Sie ist für das Projekt passend, da jedes Element direkt weiß, wer seine direkten Nachbarn sind. Wenn neue Elemente an einer Stelle dazukommen, müssen nur Next und Prev neu ausgerichtet werden. Eine Node enthält die Data und zeigt mit einem Pointer auf die nächste Data. Sie ist um Beziehungen in Programmen darzustellen optimal. Ein Nachteil ist, dass bei einer Stelle weiter hinten in der Liste die komplette Liste durchlaufen werden muss, um genau dort ein Element einzufügen.

3 Neo4j

Neo4j ist eine Graphdatenbank, die durch Beziehungen aneinander gekoppelt wird und durch Typen beziehungsweise Labels gruppiert werden kann. Dadurch entsteht eine Abhängigkeit zwischen den Daten. Es wird erst ein Knotentyp bestimmt, der sagt, zu welcher Kategorie der Knoten gehört, zum Beispiel Person oder Essen. Danach folgen die Eigenschaften, in denen ein Value als String oder Integer stehen kann. Man ist nicht auf eine Eigenschaft beschränkt.

```
create (l:Linux {challenge:"Linux CLI", keyword:"cli", keyword:"linux"}) return l
```

Wenn man zwei Challenge-Node's erstellt hat, kann man sie mit einer Beziehung verknüpfen.

```
create (l)-[:linux]->(12)
create (1)<-[:linux]-(12)
```

Man könnte beim Erstellen von Edges auch neue Properties anlegen.

```
create (l)-[:linux{os:[Linux]}}->(12)
```

Die Edges können auch in beide Richtungen zeigen, was bei einer LinkedList der Fall ist. Es gibt auch die Möglichkeit, die Edges zu benennen, zum Beispiel mit `linux`, um die Beziehung zu verdeutlichen.

4 LaTeX

LaTeX ist ein Textsatzsystem, mit dem man Dokumente sauber und ordentlich aufbauen kann. Man schreibt dabei nicht direkt in das fertige Layout, sondern benutzt Befehle, aus denen später ein PDF erstellt wird. LaTeX basiert auf TeX und wurde von Leslie Lamport weiterentwickelt. Es ist vor allem dafür da, wissenschaftliche und technische Dokumente zu erstellen. Der Vorteil ist, dass man sich mehr auf den Inhalt konzentriert und LaTeX den Rest vom Layout übernimmt. Dadurch sieht das Dokument am Ende meistens sehr sauber aus. LaTeX arbeitet mit einer `.tex`-Datei. In diese Datei schreibt man Befehle, und daraus wird später ein PDF kompiliert. Der Aufbau ist immer ungefähr gleich:

```
\documentclass{article}

\usepackage[utf8]{inputenc}
\usepackage[ngerman]{babel}

\begin{document}

\section{Einleitung}
Dies ist mein LaTeX Beispiel.

\end{document}
```

In diesem Beispiel haben wir die Dokumentenklasse `article` gewählt, die für einfache Artikel geeignet ist. Es gibt auch andere Klassen wie `report` oder `book`, die für längere Dokumente gedacht sind.